arXiv:2401.00134v1 [cs.DC] 30 Dec 2023

Unicron: Economizing Self-Healing LLM Training at Scale

Tao He[1], Xue Li[1], Zhibin Wang[1,2], Kun Qian[1], Jingbo Xu[1], Wenyuan Yu[1], Jingren Zhou[1]

[1]*Alibaba Group,* [2]*Nanjing University*

unicron@alibaba-inc.com

Abstract

Training large-scale language models is increasingly critical in various domains, but it is hindered by frequent failures, leading to significant time and economic costs. Current failure recovery methods in cloud-based settings inadequately address the diverse and complex scenarios that arise, focusing narrowly on erasing downtime for individual tasks without considering the overall cost impact on a cluster.

We introduce Unicron, a workload manager designed for efficient self-healing in large-scale language model training. Unicron optimizes the training process by minimizing failure-related costs across multiple concurrent tasks within a cluster. Its key features include in-band error detection for real-time error identification without extra overhead, a dynamic cost-aware plan generation mechanism for optimal reconfiguration, and an efficient transition strategy to reduce downtime during state changes. Deployed on a 128-GPU distributed cluster, Unicron demonstrates up to a 1.9× improvement in training efficiency over state-of-the-art methods, significantly reducing failure recovery costs and enhancing the reliability of large-scale language model training.

1 Introduction

Large language models (LLMs) like ChatGPT [37], BERT [8], BLOOM [43], Llama [47], and Llama-2 [48] are widely used in various real-world applications [5,8,20,39], drawing significant attention from both academia and industry for their role in advancing natural language processing and AI. These comprehensive models, often comprising billions of parameters, are trained on large-scale GPU clusters [40,44]. To facilitate their training on thousands of GPUs, distributed frameworks like Megatron-LM (Megatron) [34,44] and DeepSpeed [41] have been developed, offering efficient parallelization and optimization.

With advanced models demanding extensive computational capabilities, cloud platforms provide a practical and cost-effective approach to Large Language Model (LLM) training by enabling the straightforward provisioning of GPU-rich clusters as required. Notable cloud providers such as Amazon Web Services [2], Google Cloud Platform [14], Microsoft Azure [28], and Alibaba Cloud [7] provide specialized services tailored to distributed model training. However, training failures remain a common challenge, primarily due to the considerable volume of deployed resources and extended training durations, as underscored in several recent studies [19,43,49,50,54]. Our analysis of one of the major cloud platforms, referred to as Alibaba Cloud for clarity in this paper, echoes this observation. We found that the failure rates for LLM training tasks can skyrocket to 43.4% for the top 5% most resource-intensive tasks, as illustrated in Figure 1.

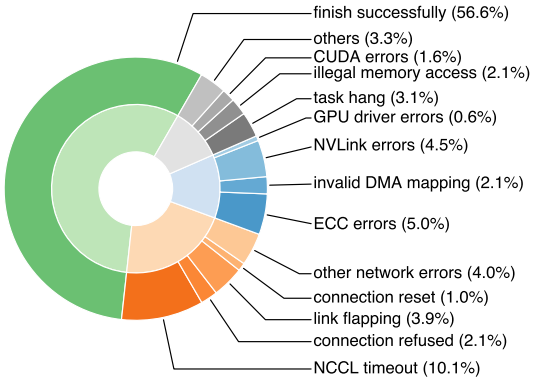


Figure 1: Distribution of task termination statistics.

Figure 2 delineates the divergent paths of recovery in the event of errors during a GPT-3 training exercise on Alibaba Cloud, leveraging a fleet of 256 NVIDIA H800 GPUs within the Megatron framework. For transient faults, which constitute 73% of all errors and are typically remediable by restarting the system, the recovery trajectory may involve a system hang lasting up to 30 minutes – stemming from the `all-reduce` communication timeout. This delay is followed by task termination and a succession of steps including a 9-minute wait for task resubmission, a 14-minute span for environment and CUDA configuration, and a final 15-minute recomputation phase. Collectively, this results in a downtime of 68 minutes. Conversely, hardware faults – which precipitate the need for node drainage in 37% of cases – set off a more

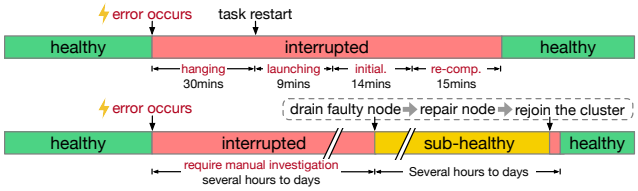


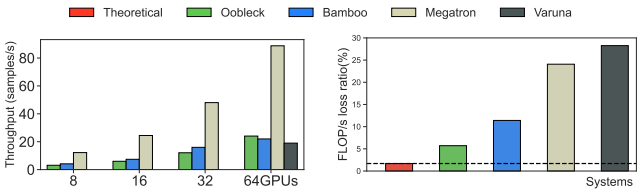
Figure 2: Training process with manual failure recovery.

labor-intensive recovery entailing manual fault identification, node drainage, down-scaling of Megatron's configuration, and checkpoint realignment before resumption of training. This manual labor can extend the 'interrupted' state for several hours to days, relegating the system to a 'sub-healthy' state of compromised capacity. The scarcity and high cost of GPU resources exacerbate the financial stakes of these failures, as any inefficiency or idle time translates directly into considerable economic loss by squandering invaluable training time.

Despite these challenges, a range of methods have been developed to mitigate the impacts of training failures. However, existing methods often fail to provide comprehensive solutions, as they primarily focus on individual aspects of LLM training failure handling. For instance, some studies [9, 31, 35, 49, 50] concentrate on the checkpointing process with the aim of reducing interruptions durations. Others propose elastic training and scheduling strategies [3,16,19,24,27,45,51,52], while additional works [3,46] explore the use of redundant computation or hot spares to prevent task interruptions. However, these solutions typically overlook the complexities of failures, which require an all-encompassing recovery strategy – from quick error detection to swift recovery to a healthy state and seamless transitions between task running states when nodes are draining or joining. Consequently, there is a significant gap in the current methodologies which necessitates a more holistic approach.

Moreover, systems specialized for training resilience such as Oobleck [19], Bamboo [46], and Varuna [3] operate at a fraction of Megatron's efficiency, leading to a scenario where resources are expended but not effectively utilized for training, as demonstrated in Figure 3a. This inefficiency becomes even more pronounced when considering the throughput losses due to failures. As Figure 3b reveals, a mere 2% downtime can lead to throughput losses that are threefold or greater than the optimal scenario (compared with their own respective implementations). Such discrepancies indicate a misalignment in fault recovery objectives: *the essence lies not in sustaining training processes through failures but in economizing the entire training to minimize lost throughput*.

Unicron. To address existing limitations in failure recovery during LLM training on cloud platforms, we introduce Unicron, a distributed workload manager built with Megatron. Unicron is designed to enhance the training process by adopting a comprehensive strategy that focuses on minimizing the total cost of failures. This is achieved through a combination of efficient error detection, seamless transitions in system



(a) Throughput of training without failures.

(b) The FLOP/s reduction caused by failures.

Figure 3: Throughput and FLOP/s reduction of training the GPT-3 7B model on a cluster of 64 GPUs for 7 days with 10 node fault errors occurred during the period. (a) The throughput is the number of samples the system can process per second. (b) The theoretical reduction is the ratio of hardware resources due to the unavailability during failures. For each system, the reduction is the percentage of lost FLOP/s compared with the ideal FLOP/s it can achieved assuming no failure happens.

states, and effective management of sub-optimal conditions.

Unicron's key features include inheriting all optimization techniques from Megatron, ensuring efficient training task execution. It preserves strict optimizer semantics, guaranteeing exact parameter updates without resorting to asynchronous or approximate methods during recovery from faults. The system's non-intrusive design facilitates easy adaptation to existing workloads and seamless integration with future updates to Megatron. Furthermore, Unicron's self-healing capabilities enable it to efficiently detect and recover from a variety of failures. Its multi-tasking approach maximizes overall resource utilization across multiple tasks within a cluster, embodying economic efficiency in large-scale LLM training. Our key contributions can be outlined as follows:

- We strategized the architectural design of Unicron (Section 3), integrating it with Megatron [34] for its high performance training capabilities and inserting new features to streamline failure recovery.
- We developed efficient error detection and handling approaches (Section 4), enabling Unicron to promptly identify failures during execution and initiate suitable corrective actions tailored to the particular failure causes.
- We formulated a cost-aware plan generation mechanism (Section 5), aiding Unicron in configuring the most optimal plan. This mechanism is informed by a model considering the multiplicity of tasks within the cluster.
- We introduced a transition strategy (Section 6), which minimizes system transition durations by exploiting partial results from ongoing training iterations and leveraging the nearest principle for state migration.
- We conducted extensive experiments, using a myriad of representative training tasks on a cluster comprised of 128 GPUs (Section 7). These experiments revealed that Unicron markedly reduces costs associated with recovery from failures resulting in an enhancement of up to 1.9× in overall training efficiency.

2 Background and Opportunities

This section provides a comprehensive overview of the architecture underpinning distributed LLM training, delves into the prevailing statistics on system failures, and evaluates the current strategies implemented for failure recovery. Furthermore, it elucidates the opportunities that these challenges present, laying the groundwork for transformative approaches in resilience and efficiency within this domain.

2.1 LLM Training

Training Frameworks. The scale of data and computational intensity required for training Large Language Models (LLMs) necessitates the adoption of distributed training frameworks. Recent benchmarks, such as the MLPerf results [29], showcase the formidable capabilities of NVIDIA's accelerated computing platforms. Specifically, leveraging 10,752 NVIDIA H100 Tensor Core GPUs coupled with Quantum-2 InfiniBand networking, NVIDIA has achieved the impressive feat of training a GPT-3 model with 175 billion parameters over 3.7 trillion tokens in just eight days [30]. The cornerstone of this achievement is Megatron [44] – a robust transformer architecture by NVIDIA that integrates a slew of advanced optimizations, ensuring over 50% of FLOP/s utilization of available computational resources, a benchmark substantiated by Figure 3a. This blend of performance and efficiency positions Megatron as a venerated choice for LLM training [10, 13, 43].

Parallelism Approaches. The quest for efficiency in distributed training has given rise to various parallelism strategies. Data Parallelism (DP) [17, 21, 25, 41] distributes the workload evenly across multiple workers, each processing a distinct subset of data. In contrast, Pipeline Parallelism (PP) [12, 15, 22, 32, 33, 41] slices the model into sequential stages, optimizing the process via micro-batches. Tensor Parallelism (TP) [34, 44], another variant, opts for vertical model partitioning. The fusion of DP, PP, and TP—termed '3D Parallelism'—along with additional techniques like sequence parallelism [23, 26] and gradient checkpointing [6, 18], enhances the adaptability and scalability of LLM training, affording a degree of natural elasticity in managing resources within the constraints of memory limitations.

2.2 Failure Statistics

In the domain of LLM training, failures are an unwelcome yet common occurrence, often arising from the immense computational loads and extended training periods required. The root causes are varied, encompassing complexities in network topology and inconsistencies in hardware reliability [42]. An examination of the failure patterns on the Alibaba Cloud platform has revealed that the most resource-intensive tasks—representing the top 5% – exhibit a startling 43.4% rate of abnormal terminations, underscoring the critical need for more resilient training systems.

Contrary to what one might expect, these failures are not continuous but sporadic. The bulk of GPUs perform without issues for the majority of the time. In a typical set of 128 GPUs, failure frequencies range from once to seven times weekly, equating to an average time between failures of over a day. This pattern of infrequent yet impactful disruptions mirrors trends reported in Meta's training of the OPT model [54].

Failure Recovery Cost. The recovery from such failures, as depicted in Figure 2, is a multi-stage process beginning with error detection and the determination of a viable configuration for resumption. The subsequent steps involve transitioning to this new configuration and then continuing training, potentially at a reduced efficiency. This process incurs distinct costs at each phase—whether managed manually or automatically—including the time lost during error detection, the effort required to reconfigure the system, and the diminished throughput in less-than-ideal operational states. These costs, which collectively impact the overall efficiency and output of the training process, can be formalized as follows:

$$C_{\text{recovery}} = C_{\text{detection}} + C_{\text{transition}} + C_{\text{sub-healthy}} \tag{1}$$

2.3 Related Work

Error Detection. Contemporary cloud services offer fundamental monitoring tools that allow for basic oversight of system operations [42]. Advanced research efforts are pushing these boundaries, advocating for more integrated and sophisticated monitoring solutions to preemptively catch errors before they escalate [49, 50].

Checkpointing. Checkpointing, a technique that periodically saves the training state, serves as a pivotal recovery mechanism in deep learning frameworks. The focus of recent research has been to streamline checkpointing—enhancing its efficiency to mitigate the time lost to recomputation and accelerate the recovery process in the event of a failure [1, 9, 31, 35, 38, 49]. Despite its benefits, the complexity of failures within distributed training systems often transcends what checkpointing alone can address.

Elasticity. In the face of failures, some systems employ elasticity – adjusting parallelism settings to prevent training interruptions [3, 16, 19, 24, 27, 45, 51, 52]. While this adaptability is advantageous for maintaining operational continuity, it may introduce additional overhead and potentially reduce throughput. Moreover, the complexity of integrating such elastic designs into high-performance systems like Megatron often poses significant challenges.

Redundant Computation and Hot Spares. Other strategies involve employing redundant computation or allocating hot spares to preempt failures [3, 46]. While these methods aim to provide a buffer against disruptions, they come with a significant economic and resource cost, highlighting the need for more efficient solutions.

Workload Managers. Workload managers SLURM [53] and Kubernetes [4] are pivotal in orchestrating general-purpose computing, offering a suite of functionalities that include

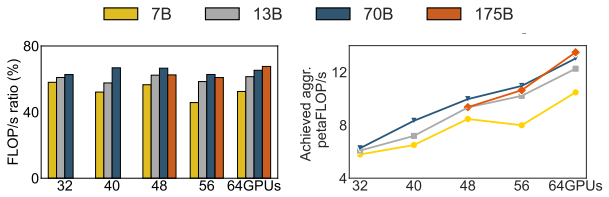


Figure 4: Achieved FLOP/s ratio and aggregate FLOP/s for training varying-sized GPT-3 models using Megatron.

cluster management, task queuing, scheduling, and efficient resource allocation. These platforms typically handle tasks as black boxes – opaque entities with predefined static configurations – submitting them into a queue for execution without intimate knowledge of their inner workings. While they are equipped with mechanisms like hot spares and automated retries for basic failure handling, they lack the bespoke features necessary for the specialized domain of LLM training. As a result, when failures occur, the intricate task of diagnosis and recovery largely reverts to the LLM training tasks themselves.

2.4 Opportunities

We have identified three key opportunities not yet fully realized by existing solutions, which could significantly enhance the management of LLM training failures.

O1: Cost-Effectiveness in Failure Management. In LLM training, failures are not just operational setbacks; they entail excessive manual interventions and wasted training time, causing significant inconvenience as highlighted in Sections 1 and 2.2. Indeed, around 73% of such failures can be rectified by simply restarting the training processes. Although this process takes about 68 minutes under default settings, sophisticated checkpointing methods like GEMINI [49] offer potential reductions in downtime. Despite the mean time between failures being relatively infrequent – ranging from a day to a week for a 128 GPU cluster – the economic implications cannot be overlooked. Resilient training systems [3, 19, 46], which show considerably less efficiency compared to Megatron in normal conditions (as shown in Figure 3a), are not economically sustainable. Relying on such systems is akin to keeping a significant portion of the training cluster idle, which is impractical, especially in larger clusters. Additionally, the use of hot spares and redundancy, while helpful for continuity, must be balanced against their economic impact to avoid resource wastage. This situation raises a critical question: *how can we alleviate the pain points associated with failures while simultaneously maximizing economic efficiency and minimizing resource wastage?*

O2: Inherent Elasticity with Non-Linear Performance. The concept of 3D parallelism introduces a remarkable degree of flexibility and elasticity into LLM training. This framework allows for adjustments in parallelism across three distinct dimensions while preserving the integrity of the training semantics. However, this elasticity is a double-edged sword. It demands meticulous management to optimize resource utilization and enhance performance. For instance, Figure 4 illustrates the variations in achieved aggregate FLOP/s and its ratio to the theoretical peak FLOP/s of a healthy system (settings detailed in Section 7.4). A notable trend is the non-linear, and sometimes non-monotonic, relationship between the number of GPUs and the achieved aggregate FLOP/s. Adding a seemingly small number of GPUs, say 8 to a 48 GPU cluster, can lead to performance dips due to the inability to directly translate the optimal configuration from a 48 GPU setup to a 56 GPU one, mainly owing to memory constraints. Additionally, tasks operating at different scales may exhibit varying levels of resource utilization. This phenomenon underscores a critical insight: simply maximizing the use of available resources doesn't guarantee peak performance. It also implies that altering parallelism settings can significantly impact the efficiency of each GPU in the cluster. The question then becomes: *how can we harness this inherent elasticity of LLM training to ensure resilient operations while simultaneously achieving high performance and efficient GPU utilization?*

O3: Rethinking from a Workload Manager's Perspective. Workload managers, positioned uniquely at the helm of cluster operations, can revolutionize the approach to LLM training failures. These systems, such as SLURM and Kubernetes, are not just for queueing and managing tasks; they provide a global perspective of the entire cluster, coupled with detailed node-specific information through agents. This comprehensive oversight allows for a holistic management approach, transcending the traditional method of treating training tasks as isolated, black-box entities in a queue. Such managers can seamlessly integrate with advanced training systems like Megatron, enhancing the efficacy of existing tools for checkpointing and monitoring. The real transformative potential lies in leveraging the inherent elasticity of LLM training tasks. By no longer confining these tasks to static configurations within queues, a workload manager can dynamically select optimal configurations, efficiently managing resources in response to real-time conditions. This strategy extends to handling the intricacies of failure recovery, guiding the system from detecting and addressing issues to smoothly transitioning back into Megatron's standard training process. This leads us to a crucial consideration: *can we innovate a workload manager that not only supports self-healing LLM training at scale but also adopts a holistic approach, maximizing economic efficiency throughout the process?*

3 System Design

Our Answer: Unicron is designed as a distributed workload manager built with Megatron, to enhance the training process by achieving economic efficiency and self-healing capabilities. The architecture of Unicron is depicted in Figure 5. The non-intrusive integration of Unicron with Megatron guarantees the preservation of both existing techniques and the future functionalities of Megatron, thereby ensuring efficient training task execution and maintaining strict semantics preservation. Fur-

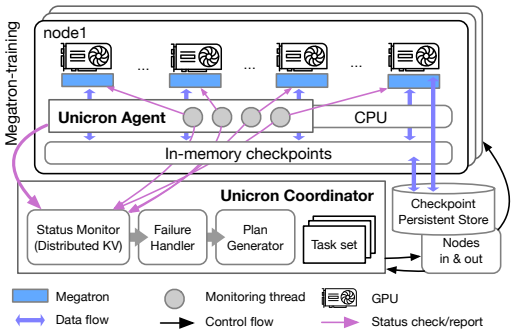


Figure 5: The system architecture of Unicron.

thermore, Unicron introduces additional components, specifically the *Unicron agent* and the *Unicron coordinator*, as well as several key techniques to enable efficient self-healing in the event of various failures during LLM training. Additionally, Unicron considers concurrent training tasks within the cluster, with the goal of improving overall resource utilization. The combination of these features ensures that Unicron maintains economic efficiency while effectively achieving self-healing capabilities. Next, we will discuss the key components and techniques employed by Unicron in detail.

3.1 Unicron Agent

The Unicron agent is a crucial component, performing several key tasks associated with a single machine in the cluster.

Error Detection. The Unicron agent establishes a persistent connection with the coordinator, which helps to ensure the continuous availability and responsiveness of the node. Besides, it assigns a dedicated monitoring thread on the CPU for each GPU. These monitoring threads closely observe the status of the GPU, capturing important events and detecting any irregularities or exceptions. This fine-grained monitoring helps in identifying potential failures or issues promptly.

Recovery Actions Execution. The Unicron agent plays a significant role in executing recovery actions inside a machine, guided by the Unicron coordinator and following a transition strategy. These actions aim to swiftly transition the system from its current state to a new state, ensuring the system is rapidly restored to a functional state.

Checkpointing. Recognizing the necessity of checkpointing for failure recovery as a bottom-up solution, Unicron incorporates a checkpointing mechanism. The Unicron agent manages the checkpointing workflow, handling in-memory checkpoints following the methodology proposed by GEMINI [49], and asynchronously transfers these checkpoints to a remote persistent storage. This hierarchical checkpointing approach guarantees the ability to restore training progress in the event of failures. It is noteworthy that our framework operates independently of research aimed specifically at optimizing the checkpointing process. Advanced mechanisms can be integrated into Unicron to reduce the overhead associated with checkpointing, as our focus lies elsewhere.

3.2 Unicron Coordinator

The Unicron coordinator leverages the information collected by the agents to make informed decisions and coordinate actions across the entire cluster. Its responsibilities include:

Consolidation of Process Status. The Unicron coordinator utilizes a distributed key-value store (implemented using etcd [11]) referred to as the status monitor, to consolidate and store the process statuses reported by the monitoring threads from each agent, allowing for a comprehensive view of the system's health.

Error Handling. Upon detecting abnormal status from the status monitor, the Unicron coordinator assesses the situation to determine the correct response. It then directs the agent to implement the necessary actions, ensuring that these actions are well-coordinated and synchronously executed across the workers involved.

Reconfiguration Plan Generation. The Unicron coordinator plays a crucial role in developing an optimal reconfiguration plan when necessary, such as in cases of node faults, node integration, task finished, or task launched within the cluster. This plan is guided by a comprehensive model that considers all ongoing tasks within the cluster, with the primary objective of minimizing any potential decline in training efficiency.

Training Task Management. The Unicron coordinator is responsible for managing the training tasks within the cluster. It keeps a watch on the status of each task, coordinates task submission and termination with the cloud service, utilizing a task set for tracking. Each task is associated with the minimum computational requirements, as well as a weight assigned by the user to model its priority. The weights can represent task priority or the unit price / quota the users willing to pay for each unit of useful FLOP/s. These weights and computational requirements serve as essential inputs for the coordinator to determine the optimal reconfiguration plan for distributing available resources among the tasks.

Other External Interactions. In addition to its primary responsibilities, the Unicron coordinator also handles other external interactions. These interactions encompass a range of activities such as alerting maintenance personnel of failure incidents, acknowledging node recovery updates, incorporating newly provisioned nodes from the cloud service, and others.

3.3 Key Techniques

Unicron incorporates several key techniques to address the three types of costs related to failure recovery. Firstly, the cost of detecting errors, denoted as $C_{\text{detection}}$, is managed by the error detection module described in Section 4.1. This module leverages four distinct detection methods to ensure failures are identified promptly and accurately, while avoiding extra overhead on the training process. Next, the actions taken to respond to failures, including reattempting in-place, restarting the training process, and reconfiguring the cluster, contribute to the transition cost, $C_{\text{transition}}$. Unicron seeks to minimize this cost through the adoption of a rapid transition strategy,

which is explained in Section 6. Lastly, the cost of sub-healthy, referred to as $C_{\text{sub-healthy}}$, is associated with reduced resource utilization and training efficiency due to sub-optimal configurations. This cost can have a prolonged impact on overall system performance. To mitigate it, Unicron utilizes a formulated model designed to determine the most effective configuration plan. The development and solution of this model are discussed in detail in Section 5.

4 Error Detection and Handling

In this section, we present the design for error detection and handling strategies in Unicron. Effective error detection methods are essential for minimizing detection costs, while tailored handling strategies ensure appropriate measures for failure recovery. Table 1 summarizes the four detection methods employed by Unicron, along with our subjective classification of error statuses based on their severity levels. The severity levels represent the impact of the error on the training process, ranging from SEV1 (most severe) to SEV3 (least severe), and are used to determine the appropriate handling strategy.

Table 1: Detection methods and severity levels of errors.

Detection method	Error status	Severity
Node health monitoring	Lost connection	SEV1
Process supervision	Exited abnormally	SEV2
Exception propagation	Connection refused/reset	SEV3
	Illegal memory access	SEV2
	ECC errors	SEV1
	Invalid DMA mapping	SEV1
	CUDA errors	SEV2
	NVLink errors	SEV1
	GPU driver errors	SEV1
	Other network errors	SEV3
	Other software errors	SEV2
Online statistical monitoring	NCCL timeout	SEV3
	Link flapping	SEV3
	Task hang	SEV2
	Other software errors	SEV2

4.1 Error Detection

Unicron utilizes in-band error detection by continuously monitoring the real-time status of each training processes. This is done through the monitoring threads of the agent, which track training progress, exceptions and communication timeouts. The agent is launched at the beginning of the training process and operates concurrently with it. It enables the system to promptly identify any irregularities or exceptions. Compared to other solutions that rely on out-of-band monitoring like cloud monitoring services, this method provides a significant advantage in terms of efficiency and accuracy. Furthermore, the monitoring threads operate on the CPU, ensuring that they introduce no extra load on the GPU, which carries out the primary training workload.

Node Health Monitoring. The persistent connection maintained between the Unicron agent and the Unicron coordinator guarantees node availability. If this connection is lost, the node is marked as unavailable, and a SEV1 failure is triggered.

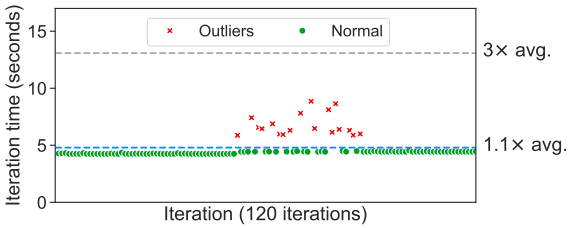


Figure 6: Completion time per iteration.

Process Supervision. The Unicron agent has a monitoring thread for each GPU to watch the training process. Should a process terminate unexpectedly, this thread signals a SEV2 failure to the coordinator for resolution.

Exception Propagation. Exception handling is crucial for the prompt detection of incidental errors, such as ECC errors, NVLink errors, CUDA errors, and others. These errors are immediately identified once the GPU issues an exception, which is then captured by the monitoring thread and reported to the coordinator.

Online Statistical Monitoring. Unicron leverages online statistical monitoring to detect certain errors like NCCL timeout, TCP timeout, and task hangs, where notifications are delayed. For instance, a delay of up to 30 minutes may occur before a NCCL timeout error is raised, as shown in Figure 2. Although the timeout threshold can be adjusted, it is challenging to determine the appropriate value, as it is highly dependent on the system configuration and the workload.

The monitoring threads implement online statistical monitoring to detect these errors. Under normal training conditions, which proceed periodically, the statistics of iterations should reveal a relative consistency when the system is set to a particular configuration, as Figure 6 shows with green dots representing iteration completion times for training a GPT-3 175*B* model on 256 NVIDIA H800 GPUs. While minor fluctuations may occur due to network variations and congestion, they typically stay within a reasonable margin indicated by the blue line (i.e., 1.1× the average iteration time). The red dots illustrate instances where a network switch has been deliberately turned off, leading to a marked increase in completion time; yet, the training process manages to persist. If the waiting time surpasses the threshold denoted by the grey line, this confirms a failure, requiring immediate recovery measures. Empirical evidence suggests that setting the failure threshold at 3× the average iteration time, achieves a practical balance between efficiency and accuracy.

4.2 Error Handling

When any abnormal status is detected, the Unicron coordinator is notified and proceeds to take appropriate actions. The first step in handling a failure is to classify the collected status based on its severity level, as outlined in Table 1. This classification (indicated by ① to ③) helps determine the appropriate recovery strategy, which corresponds to one of three specific

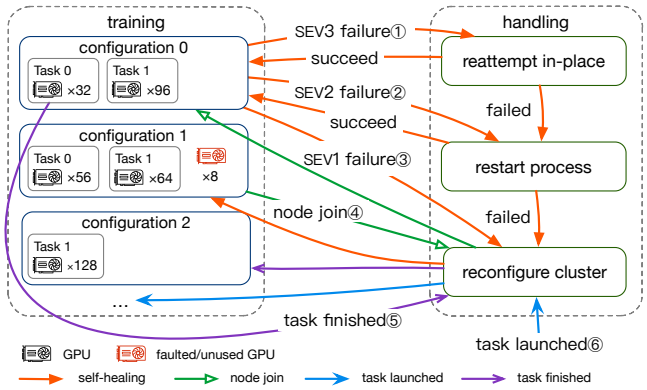


Figure 7: The error handling workflow in Unicron.

actions, following the guidance of Figure 7. Additionally, Figure 7 includes other triggers (indicated by ④ to ⑥), to ensure the smooth operation of the training process.

Reattempt In-place. The initial attempt to mitigate a failure involves retrying the operation where it failed, assuming there are no indications of underlying software or hardware issues, which is classified as SEV3 failure ①. This method is effective for addressing issues like temporary connection problems (e.g., link flapping or connection refused/reset). If the reattempt succeeds, the training process immediately proceeds as normal. Otherwise, the issue is upgraded to a SEV2 failure.

Restart Process. SEV2 failures ②, such as CUDA errors or illegal memory accesses, are resolved by restarting the training process on the affected node. The system configuration, including the number of GPUs, parallelism settings, and process ranks, remains unchanged. And the training states are recovered from either other data parallel replicas or the latest checkpoint, as detailed in Section 6. If the restart attempt fails, the severity of the failure is upgraded to SEV1.

Reconfigure Cluster. In the event of a SEV1 failure ③, the Unicron coordinator isolates the failed node and initiates a reconfiguration procedure. This happens when the coordinator receives notifications of GPU node faults or if the system fails to recover from a SEV2 failure. When a previously failed and unused node is successfully recovered and becomes available again, or when a new node is allocated, it can *join* ④ the ongoing training process. In such cases, the Unicron coordinator initiates the cluster to enter the reconfiguration process, which allows the system to adapt and incorporate the additional available resources. Additionally, the reconfiguration process can also be triggered when an existing task is *finished* ⑤ or a new task is *launched* ⑥. This is because the optimal configuration plan may differ from the current configuration, requiring the reallocation of resources to achieve the global optimal training efficiency of the cluster.

5 Optimal Reconfiguration Plan Generation

This section introduces a method for generating an optimal reconfiguration plan to efficiently distribute tasks across GPUs within a distributed training cluster. The formulation of this plan is critical as it directly influences the sub-healthy cost $C_{\text{sub-healthy}}$. To capture these costs, we define the metric of WAF, representing the weighted achieved aggregate FLOP/s, and formulate an optimization problem to minimize the costs incurred during sub-healthy and transition states.

5.1 Model Formulation

With n workers available in the cluster for distributed training and m tasks to be trained, our goal is to fully utilize the computation capacity of the resources while meeting the requirement of each running task. A GPU card is considered as a worker by default within the scope of this problem. Before delving into our model, we first define the metric to measure the training efficiency of a task.

WAF. The WAF of a task measures the weighted achieved aggregate FLOP per second. Formally, we define a function $F : \mathbb{N} \times \mathbb{N} \to \mathbb{R}$, where $F(t,x)$ represents the WAF of task t when x workers assigned to it.

As implied by the name, the most important part of WAF is the *achieved aggregate FLOP/s*, denoted as $T(t,x)$, for a given task t with x workers. Notice the $T(t,x)$ reflects the optimal performance of the task, which is obtained by tuning the complex combination of parallelism hyperparameters and other optimization settings. To address this, we rely on calibrating tasks on the given GPU cluster and leverage automatic execution plan generation techniques [55] to estimate the optimal parallelism settings and associated $T(t,x)$.

In addition to the achieved aggregate FLOP/s, we further integrate two additional factors to model the minimum computational requirements and task priorities. *Requirement condition* ($T_{\text{necessary}}(t)$) represents the minimum required resources for a task given. A task will only be scheduled if the available resources can satisfy this requirement. *Task weight* ($w(t)$) is used to model the priority of a task. Tasks with higher priority are assigned higher weights. By default, we set $w(t) = 1$ for all tasks. This weight factor allows the user to adjust the relative importance of different tasks in the problem, with the recommended values for $w(t)$ range between 0.5 and 2.0.

Accordingly, the WAF is defined as follows:

$$F(t,x) = \begin{cases} w(t) \cdot T(t,x) & \text{if } (t,x) \vdash T_{\text{necessary}}(t), \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Here, $F(t,x)$ is calculated as the product of the task's weight, $w(t)$, and its achieved aggregate FLOP/s, $T(t,x)$, when (t,x) satisfies the necessary requirements, $T_{\text{necessary}}(t)$. Otherwise, if it falls short of this threshold, the WAF of the task is considered to be zero.

Optimization Objective. The formulation of the optimization problem for plan generation is centered around a trade-off: maximizing the WAF of the cluster post-reconfiguration while minimizing the impact on GPUs during the transition.

$$
\begin{aligned}
&\arg\max_{x'_1,\ldots,x'_m} \quad \sum_{i=1}^{m} G(t_i, x'_i), \\
\text{where} \quad & G(t_i, x'_i) = F(t_i, x'_i) \cdot D_{\text{running}}(n') \\
& \qquad - F(t_i, x_i) \cdot \mathbf{1}(t_i, x_i \to x'_i) \cdot D_{\text{transition}}, \\
\text{subject to} \quad & \sum_{i=1}^{m} x'_i \le n'.
\end{aligned} \tag{3}
$$

Prime notation (') differentiates states before and after reconfiguration. The subscript i denotes the task identifier, t_i represents the i-th task, and x_i the number of workers initially assigned. The constraint ensures that the workers assigned across all tasks do not exceed the total available in the cluster. The goal is to maximize the cluster's cumulative reward, represented by the sum of individual task rewards $G(t_i, x'_i)$.

The primary term, $G(t_i, x'_i)$, reflects the reward when the cluster is operational post-reconfiguration, calculated as the WAF of task t_i with x'_i workers, multiplied by the expected run duration $D_{\text{running}}(n')$. This duration is contingent on the operational condition of GPUs and the cluster size; a larger pool of GPUs implies a higher likelihood of failure, potentially shortening the run duration.

The penalty term, $F(t_i, x_i) \cdot \mathbf{1}(t_i, x_i \to x'_i) \cdot D_{\text{transition}}$, captures the WAF loss during transition, where $D_{\text{transition}}$ is the estimated duration of this period. The indicator function $\mathbf{1}(t_i, x_i \to x'_i)$ activates when there is a change in the number of workers assigned to task t_i or if a worker fault occurs:

$$
\mathbf{1}(t_i, x_i \to x'_i) = \begin{cases} 1 & \text{if } x_i \neq x'_i \text{ or a worker of } t_i \text{ faults}, \\ 0 & \text{otherwise.} \end{cases} \tag{4}
$$

This term discourages frequent reconfiguration, especially for healthy tasks, by factoring in the WAF that could have been achieved by unaffected GPUs.

The significance of each term is context-dependent. In stable clusters with fewer faults, maximizing WAF takes precedence, focusing on the reward during healthy operation. Conversely, in larger or less reliable clusters where faults are more common, the penalty term becomes more critical, emphasizing the need to limit the scope of reconfigurations and maintain as many tasks running as possible.

5.2 Solving Algorithm

This problem can be solved through the dynamic programming algorithm, in which we define the state as $S(i, j)$, representing the maximum value of the first i tasks with j workers. Accordingly, the state transition equation is given by:

$$
S(i, j) = \max_{k=0}^{j} \{S(i-1, j-k) + G(t_i, k)\} \tag{5}
$$

This equation implies that the maximal reward of the first i tasks with j workers can be transited from the maximal reward of the first $i-1$ tasks with $j-k$ workers plus $G(t_i, k)$, where k denotes the number of workers assigned to the i-th task. We set $S(0, j) = 0$ for $j > 0$, to initialize the states. The optimal value of Equation 3 can be obtained from $S(m, n')$. To obtain the optimal assignment strategy specifically, we can trace back the state transition process.

Complexity. The time complexity of this algorithm is $O(mn^2)$, where m represents the number of tasks and n represents the cluster size. In practice, both m and n remain moderate in size, making this algorithm efficient to execute. Furthermore, the Unicron coordinator can pre-compute and prepare the lookup table for the dynamic programming algorithm in advance, taking into account potential failure scenarios of any task or joining node. This allows for one-step advancement from the current configuration. Once reconfiguration is required, the Unicron coordinator can directly retrieve the optimal assignment strategy from the lookup table, thereby reducing the time complexity to $O(1)$.

6 Transition Strategy

In this section, we delve into the implementation of transitioning a training task to a new configuration. We first review the training process of an iteration in Megatron to identify the maximal possible partial results that can be reused. Next, we present the adaptive resumption process according to failure scenarios. This resumption can leverage the partial results to finish the current iteration. Finally, we discuss the transition process for a task that is instructed to reconfigure. In this process, we aim to minimize the state migration cost by leveraging the nearest available states.

6.1 Iterations in Megatron

We first review the training process of an iteration in Megatron, which is depicted in Figure 8. In an iteration, a *global-batch* of samples will be forward passed through the model, and the gradients will be accumulated until the completion of the backward pass. For distributed training, a global-batch is further partitioned into multiple *micro-batches*, e.g., 8 micro-batches in the figure.

Corresponding to the parallelism of the model, a rank in distributed data parallelism (DP) is responsible for a subset of the micro-batches. With the DP degree of DP, and B micro-batches in a global-batch, each DP rank is responsible for $k = B/DP$ micro-batches. Therefore, we utilize two dimension of indexes to denote the j-th micro-batch in the i-th DP rank, i.e., $grad_{i,j}$. The aggregated gradient for the global-batch, denoted as $grad$, is computed as follows:

$$
grad = \underbrace{\sum_{i=1}^{DP}}_{\text{all-reduce}} \underbrace{\sum_{j=1}^{k} grad_{i,j}}_{\text{accumulation}} \tag{6}
$$

Within a rank of DP, a rank of PP handles a subset of the layers in the model, a.k.a., stage. As shown in the figure, the micro-batch 1 is distributed to the DP 1, it first forward passes through the PP 1 then the PP 2, and the backward pass reverses

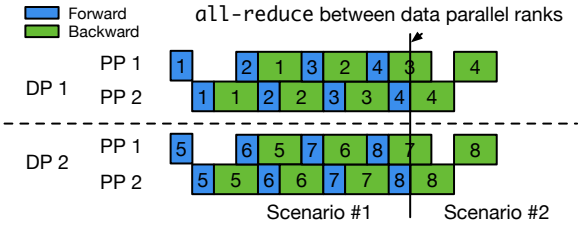


Figure 8: Timeline of data and pipeline parallelism.

the order. The PP pipeline processes the micro-batches within a DP rank and accumulate the gradients until the completion of the backward pass for the last micro-batch. Subsequently, the parameters are updated by averaging the gradients from all DP ranks through the `all-reduce` operation. Notice, that the tensor parallelism (TP) can be further employed within each PP rank, which is not shown in the figure as it does not impact our strategy.

6.2 Resuming from a Failed Iteration

Failures can disrupt training at any point, but leveraging partial results from completed micro-batches within the current global-batch can mitigate recomputation costs. Thanks to the gradient accumulation in micro-batch iterations in the distributed data parallelism, as shown in Equation 6, a state-driven approach has been devised to resume training from a failed global-batch iteration.

Failures may occur in two scenarios: prior to the `all-reduce` operation (scenario #1) or subsequent to it (scenario #2), as depicted in Figure 8. Unicron uses a micro-batch iteration scheduler that monitors and synchronizes progress, enabling the training process resume seamlessly from the point of failure. It also redistributes micro-batches from the failed DP rank to others in a round-robin fashion, ensuring the integrity of the training process.

Scenario #1. Before initiating the `all-reduce`, each DP rank manages its own accumulated gradients and independently performs forward and backward passes for micro-batches assigned to it. Unicron tracks each DP rank's progress through forward and backward passes. In the event of a failure, the resumption process consists of the following steps: 1) pausing the training process, 2) re-establishing the network connection among healthy workers, 3) redistributing the micro-batches owned by the failed DP rank, and 4) resuming the training process. After redistributing, each remaining DP rank now owns $k' = k + k/(DP-1)$ micro-batches. Consequently, the aggregated gradient for the global-batch is computed as follows:

$$grad = \sum_{\substack{i=1 \\ i \neq x}}^{x-1} \left(\underbrace{\sum_{j=1}^{k} grad_{i,j}}_{current} + \underbrace{\sum_{j=k+1}^{k'} grad_{i,j}}_{redistributed} \right) \tag{7}$$

Scenario #2. In the event of a failure occurring after the `all-reduce` operation started, the response depends on whether the failed worker's gradients have been reduced, due to pipeline parallelism inside each DP rank. 1) If the worker's gradients were already reduced, the worker can be omitted, as the aggregated gradient is already accounted for by the other DP replicas. Training proceeds uninterrupted. 2) Conversely, if the gradients from the failed worker were not yet reduced, Unicron ensures semantic correctness by redistributing the failed DP rank's micro-batches to remaining DP ranks, similar to scenario #1, and recomputing the aggregated gradient. Unfortunately, different from scenario #1 where no gradient has been reduced, there are partial gradients (segmented on the stage/layer) that have already been reduced and we need to distinguish them from the rest unreduced gradients. Accordingly, when recomputing the redistributed micro-batches, we only need to recompute the unreduced gradients and ensure the reduced gradients are not overwritten. It's worth noting that the `all-reduce` operations solely take place at the end of a global-batch iteration and occupy a minor fraction ($< 2\%$ when training the GPT-3 175B model using 128 GPUs) of the iteration time. Therefore, the likelihood of a failure occurring after the initiation of the `all-reduce` operation remains relatively low.

6.3 Transitioning to the New Configuration

Once the parameter updates for the ongoing iteration are completed, or the problematic nodes have been fixed and reintegrated into the cluster. Unicron instructs the workers to transition to the new configuration planned by the plan generator. Next, we investigate the major problem in transitioning a given task: how to migrate the training states to the new configuration.

According to Equation 4, there are two kinds of tasks requiring transitioning: 1) a worker of the task is faulted, and 2) the task is instructed to scale in or out. Regarding the first case, Unicron follows the nearest principle to minimize the state migration cost. Unicron initially attempts to request the state of the healthy rank of DP, since the state is already replicated in each DP rank. If replication cannot be fulfilled, Unicron resorts to loading the necessary data from the hierarchical checkpoints [49]. Experience from GEMINI suggests that it is highly likely that the in-memory checkpoint is available, avoiding slower access from remote storage and enabling quick resumption of training. In contrast, if the involved task is the one that runs normally, complete parameters and optimizer states must already be present in the cluster. Unicron then requests the workers to proactively replicate the state from existing workers. Notice, that different workers issue replication requests simultaneously to minimize their own transition period.

7 Evaluation

In this section, we evaluate Unicron's performance using a range of workloads and failure scenarios. We first conduct micro-benchmarks to measure Unicron's error detection time, transition time, the effective throughput and the WAF metric.

These tests showcase how effectively Unicron's techniques tackle failure recovery challenges. Additionally, we compare the overall training efficiency of Unicron's with that of established baselines under diverse failure traces, in a multi-task environment over a specified period.

Table 2: The time to detect different kinds of failures.

Case	Method	Unicron	w/o Unicron
1	Node health monitoring	5.6 seconds	5.7 seconds
2	Process supervision	1.8 seconds	D_{timeout}
3	Exception propagation	0.3 seconds	D_{timeout}
4	Online statistical monitoring	$3 \times D_{\text{iter}}$	D_{timeout}

D_{iter}: the average time of one training iteration, typically within 1 minute.
D_{timeout}: the timeout threshold of Megatron, 30 minutes by default.

7.1 Experimental Setup

Platform. All experiments are conducted on the Alibaba Cloud platform. For the evaluation, we utilize a total of 16 instances, with each instance equipped with 8 NVIDIA A800 (80GB) GPUs, 96 CPU cores, and 1,600 GB of CPU memory. The 8 GPUs inside each instance are interconnected through NVSwitch, and each instance is connected to others via four 200Gbps Ethernet NICs. We use Alibaba Cloud's cloud filesystem service as the remote persistent storage for checkpointing, which supports a maximum throughput of 20 GB/s. For the implementation of Unicron, the software versions used is Megatron v23.08, PyTorch v1.12.1 and CUDA v11.3.

Workloads. In our evaluation, we utilize the GPT-3 model [36] as the primary workload, due to its status as one of the largest and most popular models in the LLM domain. To represent different scales, we vary the size of the model parameters, specifically considering model sizes of 1.3 billion, 7 billion, 13 billion, 70 billion, and 175 billion parameters.

Baselines. For comparison, we have selected several representative systems as baselines. These baselines are chosen based on their availability and their ability on failure recovery. The first is Megatron (v23.08) without any additional optimizations introduced by Unicron. This baseline represents the solution of terminating the training process and restarting from the last persistent checkpoint when resources are recovered, without support for training with reduced resources. The second baseline is Oobleck [19], which is a state-of-the-art framework that adopts dynamic reconfiguration to enable resilient distributed training with guaranteed fault tolerance. Varuna [3] is included as another baseline, which enables asynchronous checkpointing and dynamic reconfiguration based on job morphing for fast recovery. Additionally, we also evaluate Bamboo [46], as the state-of-the-art solution for fault-tolerant distributed training through redundant computation. For Oobleck, Varuna and Bamboo, the latest open-source versions provided by the respective authors have been utilized.

7.2 Error Detection Efficiency

To evaluate the error detection efficiency of Unicron, we simulate four failure cases, by killing a node (case 1), killing a process (case 2), throwing an exception (case 3) during training, and triggering a performance degradation (case 4). These cases cover the most common types of failures that can occur during training, and are detected by the four detection methods implemented in Unicron. The experiments are performed using different tasks and cluster sizes, and the detection time remains relatively consistent for each case, except for case 4. In case 4, the detection time was influenced by the average duration of one training iteration (D_{iter}).

We compare the detection time of Unicron with the baseline approach, which is Megatron without integrating Unicron. For the baseline approach, the detection time is measured as the duration from the occurrence of the failure to the termination of the training task. The results in Table 2 demonstrate that the detection time of Unicron aligns with the baseline approach for case 1, while is significantly shorter for the remaining three cases.

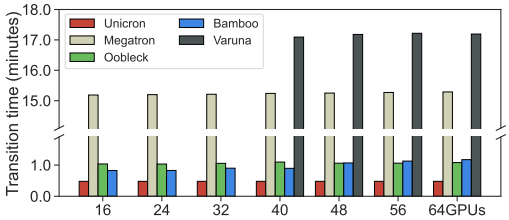


Figure 9: The transition time under failures.

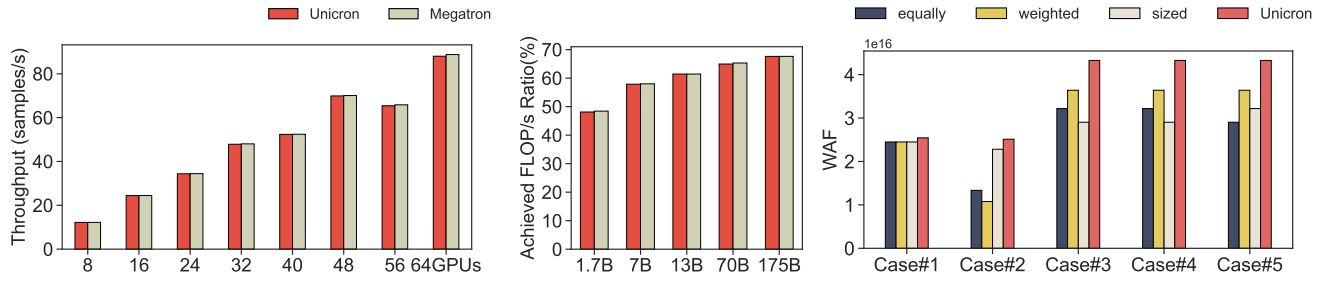
7.3 Transition Efficiency

Figure 9 presents the transition time of Unicron compared to the baselines when a SEV1 failure is detected during the training of the GPT-3 7*B* model on clusters of varying scales [1]. Missing bars indicate that the training task cannot be successfully launched for certain baselines at the specified cluster size. The transition time refers to the duration from detecting the failure to resuming training, which includes recovering the training task based on a new configuration (with a reduced number of nodes) and recomputing lost progress.

For Megatron and Varuna, the transition time required is considerably long because the training task needs to be restarted from the last checkpoint, and the progress since the last checkpoint must be recomputed [2] Oobleck and Bamboo can reduce the transition time by enabling dynamic reconfiguration, which eliminates the need to load the checkpoint and restart from it. However, their transition time still exceeds that of Unicron. Unicron further decreases the transition time by utilizing the transition technique, which maximizes the reuse of partial results from ongoing training iterations, thereby minimizing the loss caused by failures. Besides, Unicron is able to maintain a relatively stable transition time

[1] As Megatron lacks support for dynamic reconfiguration, our testing of Megatron includes the use of a hot spare node that substitutes for the failed node. Consequently, the time Megatron spends waiting for resources is not factored into the transition time measurement.

[2] Average recomputation time is 15*mins* for 30*mins* checkpoint intervals.



(a) Training throughput (single task). (b) FLOP/s ratio (single task). (c) WAF (multiple tasks).

Figure 10: Training throughput, achieved FLOP/s ratio and WAF of Unicron and baselines under different workloads.

across different cluster sizes by efficiently migrating training states between nodes, leveraging the characteristics of the parallelism approaches.

7.4 Training Throughput and WAF

In this subsection, we evaluate the training efficiency of Unicron and the baselines in terms of training throughput and WAF, considering scenarios with a single training task as well as multiple tasks on a given-size cluster, without any failures.

Comparison with Baselines (single task). The experiment involves training the GPT-3 model of different scales on clusters of varying sizes. Megatron is selected as the baseline because it exhibits significantly higher training throughput compared to other baselines like Oobleck, Varuna, and Bamboo. To ensure a fair comparison, Megatron is configured with optimal system settings including DP, PP, TP, and others, which are identical to those used for Unicron. Figure 10a presents the training throughput of training the GPT-3 7*B* model, measured in samples per second. We observe that Unicron performs on par with Megatron as it introduces no additional overhead to the normal training process and allows all the optimizations implemented in Megatron to be fully utilized. Figure 10b further validates this claim by comparing the achieved FLOP/s ratio of Unicron with that of Megatron, based on testing the GPT-3 model of varying sizes on a 64-GPU cluster.

Table 3: The tested cases in multi-task experiments.

Case		Task 1	Task 2	Task 3	Task 4	Task 5	Task 6
1	S.	7B	7B	7B	7B	7B	7B
	W.	1.0	1.0	1.0	1.0	1.0	1.0
2	S.	1.3B	1.3B	1.3B	7B	7B	13B
	W.	1.0	1.0	1.0	1.0	1.0	1.0
3	S.	7B	7B	7B	7B	7B	7B
	W.	0.5	0.8	1.1	1.4	1.7	2.0
4	S.	1.3B	1.3B	1.3B	7B	7B	13B
	W.	0.5	0.8	1.1	1.4	1.7	2.0
5	S.	1.3B	1.3B	1.3B	7B	7B	13B
	W.	2.0	1.7	1.4	1.1	0.8	0.5

S. denotes the model size, and *W.* denotes the weight of the task.

Comparison with Baselines (multiple tasks). Figure 10c compares Unicron with various baselines when training six tasks on a 128-GPU cluster, measured in the WAF metric of the cluster. Table 3 summarizes the settings for the five tested cases, which involve training multiple tasks with different sizes and priorities. Since other systems do not support generating configuration plans for multiple tasks simultaneously, we implemented several baseline strategies for comparison. The first baseline, denoted as "equally", evenly allocates computing resources among all tasks. The second baseline, denoted as "weighted", allocates computing resources based on the weight assigned to each task. The third baseline, denoted as "sized", allocates computing resources based on the model size. From the figure, it is evident that Unicron consistently achieves the highest WAF across all five cases compared to the baselines. This outcome highlights the effectiveness of the configuration plan generated by Unicron in maximizing WAF, thereby optimizing overall utilization and efficiency.

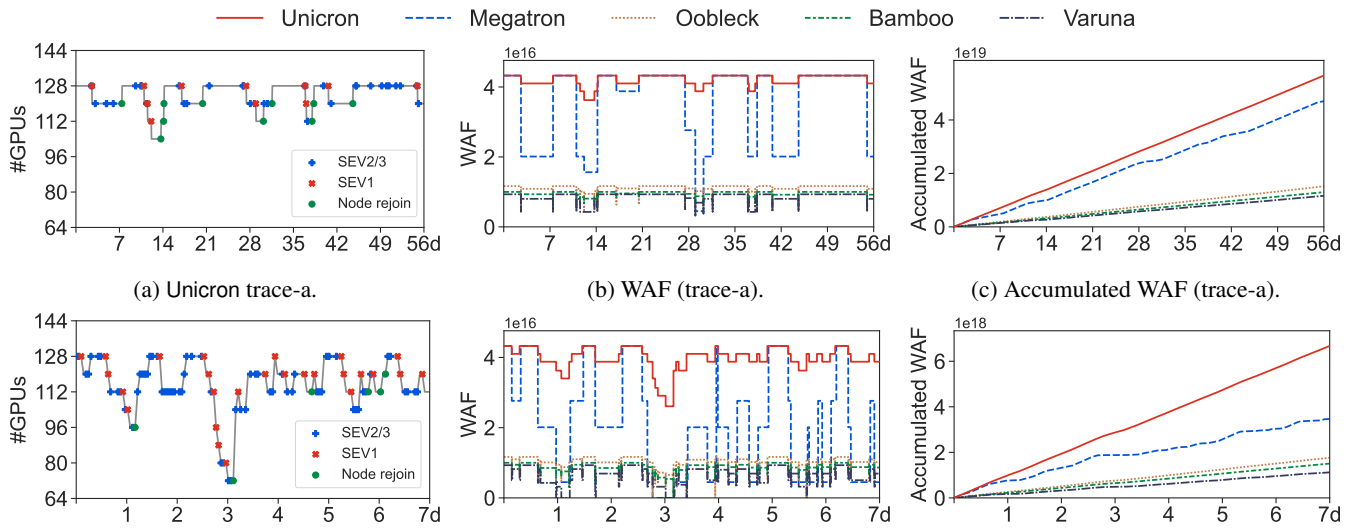
7.5 Overall Training Efficiency

Lastly, we evaluate and compare Unicron's overall training efficiency with baselines in various failure trace scenarios.

Traces. We collected a failure trace referred to as *trace-a* in Figure 11a from real-world logs we collected, as presented in Figure 1. Note that failure occurrences are considered independently for each GPU or node. The trace spans a 8-weeks period, including 10 SEV1 failures and 33 failures of other types. On the trace, the x-axis denotes the timeline of the training process, while the y-axis reflects the count of available GPUs in the cluster at any given time. It is worth mentioning that solely SEV1 failures lead to a decrease in the count of available GPUs, while other types of failures (SEV2 and SEV3) do not affect this count. Regarding SEV1 failures, the time taken for a node to recover and become available once more is determined by a uniform random selection, varying from 1 to 7 days.

Additionally, Figure 11d depicts another trace (referred to as *trace-b*), which is generated by amplifying the failure frequency of *trace-a* by a factor of 20×. The occurrence of failures during the training process is simulated using a Poisson distribution, which allows for the possibility of multiple failures happening within a given time interval. *trace-b* is designed to simulate a scenario where failures occur more frequently, thus providing a rigorous test of the systems' self-healing capabilities under extreme scenarios.

It spans a duration of 7 day and records 26 SEV1 fail-



(a) Unicron trace-a. (b) WAF (trace-a). (c) Accumulated WAF (trace-a).

(d) Unicron trace-b. (e) WAF (trace-b). (f) Accumulated WAF (trace-b).

Figure 11: Overall training efficiency (in WAF and accumulated WAF) of Unicron and baselines under different failure traces.

ures and 80 other failures. Accordingly, repaired nodes are re-joining the cluster at a similar rate to maintain a stable resource pool. This particular trace serves to replicate an environment in which failures occur more frequently, thus providing a rigorous test of the system's self-healing capabilities. In our simulation, SEV1 failures are induced by killing the agent process on the affected node, while SEV2 and SEV3 failures are triggered by either raising an exception or by killing/stalling the training process.

Workloads and Baselines. For the evaluation, the workload used is the Case#5 from Table 3. This workload involves the concurrent training of multiple tasks on a cluster with 128 GPUs. These tasks vary in size and priority, simulating real-world scenarios where different training jobs are running simultaneously. Given that the baseline systems we compare against do not possess the capability to generate configuration plans for multiple tasks, we assign the same initial configuration plan designated for Unicron to these baselines, which is considered optimal according to our proposed model. However, in the event of a failure, the baseline methods only reconfigure the task directly impacted. Moreover, should a node recover, these methods give precedence to reconfiguring the task that was first affected. In contrast, Unicron features a critical advantage over the baseline approaches: it can also reconfigure other tasks if it proves beneficial to do so.

Comparison. In Figure 11, we present a comparison analysis of the overall training efficiency between Unicron and the baselines. The efficiency metric employed is the total WAF of all tasks within the cluster, measured at successive time points. The accumulated WAF is also reported to illustrate the overall training efficiency throughout the evaluation period.

The results unequivocally demonstrate that Unicron consistently achieves the highest performance in accumulated WAF. Compared with baselines, on *trace-a*, Unicron outperforms Megatron by 1.2×, Bamboo by 4.6×, Oobleck by 3.7×, and Varuna by 4.8× in terms of accumulated WAF. It should be noted that Megatron achieves higher performance than Bamboo, Oobleck, and Varuna, which is expected given its integration of various techniques to ensure training efficiency. Conversely, Bamboo, Oobleck, and Varuna, with their primary focus on fault tolerance, do not prioritize training efficiency optimizations, resulting in lower performance.

Under *trace-b*, when failure frequency raising, Unicron outperforms Megatron by 1.9×, Bamboo by 4.8×, Oobleck by 3.8×, and Varuna by 5.8× in terms of accumulated WAF. All systems experience diminished performance under this trace due to the increased frequency of failures. Megatron suffers the most significant reduction as the recovery costs are exacerbated by the frequent failures, and it lacks specific optimizations to mitigate these additional expenses. Unicron, being built on top of Megatron, is able to fully leverage its optimizations while introducing additional optimizations for efficient self-healing. As a result, Unicron achieves significant improvements in performance compared to other baselines under this trace. These outcome underscores the effectiveness of Unicron in achieving high performance under diverse scenarios, validating its practical applicability.

8 Conclusion

This paper introduces the Unicron system as a holistic approach to address the challenges of failure recovery in training large-scale language models. By incorporating in-band error detection, a dynamic cost-aware plan generation mechanism, and a transition strategy, Unicron minimizes the overall cost of failures across multiple tasks within a cluster. As a result, Unicron demonstrates a notable increase in overall training efficiency, with performance gains reaching up to 1.9× that of state-of-the-art solutions.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.

[2] Amazon Web Services. Amazon Web Services. https://aws.amazon.com/.

[3] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 472–487, 2022.

[4] David Bernstein. Containers and cloud: From lxc to docker to kubernetes. *IEEE Cloud Computing*, 1(3):81–84, 2014.

[5] Siyuan Chen, Mengyue Wu, Kenny Q Zhu, Kunyao Lan, Zhiling Zhang, and Lyuchun Cui. Llm-empowered chatbots for psychiatrist and patient simulation: Application and evaluation. *arXiv preprint arXiv:2305.13614*, 2023.

[6] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[7] Alibaba Cloud. Alibaba cloud: Reliable and secure cloud computing services. https://www.alibabacloud.com/.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[9] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annavaram. {Check-N-Run}: A checkpointing system for training deep learning recommendation models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 929–943, 2022.

[10] Jinze Bai et al. Qwen technical report, 2023.

[11] etcd. etcd. https://etcd.io/.

[12] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. Dapple: A pipelined data parallel approach for training large models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 431–445, 2021.

[13] glm. glm. https://keg.cs.tsinghua.edu.cn/yuxiao/papers/slides-2023-ijcai-llm-glm-130-chatglm-agentbench.pdf.

[14] Google. Google cloud platform. https://cloud.google.com/.

[15] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

[16] Changho Hwang, Taehyun Kim, Sunghyun Kim, Jinwoo Shin, and KyoungSoo Park. Elastic resource sharing for distributed deep learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 721–739. USENIX Association, April 2021.

[17] Changho Hwang, Taehyun Kim, Sunghyun Kim, Jinwoo Shin, and KyoungSoo Park. Elastic resource sharing for distributed deep learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 721–739, 2021.

[18] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. Checkmate: Breaking the memory wall with optimal tensor rematerialization. *Proceedings of Machine Learning and Systems*, 2:497–511, 2020.

[19] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Oobleck: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 382–395, 2023.

[20] Andreas Jungherr. Using chatgpt and other large language model (llm) applications for academic paper assignments. 2023.

[21] Can Karakus, Rahul Huilgol, Fei Wu, Anirudh Subramanian, Cade Daniel, Derya Cavdar, Teng Xu, Haohan Chen, Arash Rahnama, and Luis Quintela. Amazon sagemaker model parallelism: A general and flexible framework for large model training. *arXiv preprint arXiv:2111.05972*, 2021.

[22] Taebum Kim, Hyoungjoo Kim, Gyeong-In Yu, and Byung-Gon Chun. Bpipe: Memory-balanced pipeline parallelism for training large language models. 2023.

[23] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems*, 5, 2023.

[24] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. Lyra: Elastic scheduling for deep learning clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems*, EuroSys '23, pages 835–850, New York, NY, USA, 2023. Association for Computing Machinery.

[25] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704*, 2020.

[26] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120*, 2021.

[27] Luo Mai, Guo Li, Marcel Wagenländer, Konstantinos Fertakis, Andrei-Octavian Brabete, and Peter Pietzuch. KungFu: Making training in distributed machine learning adaptive. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 937–954. USENIX Association, November 2020.

[28] Microsoft Azure. Microsoft Azure. https://azure.microsoft.com/.

[29] MLPerf. MLPerf. https://www.mlperf.org/.

[30] MLPerf v3.1 NVIDIA Submission. MLPerf v3.1 NVIDIA Submission. https://github.com/mlcommons/training_results_v3.1/tree/main/NVIDIA.

[31] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. {CheckFreq}: Frequent,{Fine-Grained}{DNN} checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pages 203–216, 2021.

[32] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 1–15, 2019.

[33] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. Memory-efficient pipeline-parallel dnn training. In *International Conference on Machine Learning*, pages 7937–7947. PMLR, 2021.

[34] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, 2021.

[35] Bogdan Nicolae, Jiali Li, Justin M Wozniak, George Bosilca, Matthieu Dorier, and Franck Cappello. Deepfreeze: Towards scalable asynchronous checkpointing of deep learning models. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, pages 172–181. IEEE, 2020.

[36] OpenAI. Language models are few-shot learners. https://openai.com/blog/gpt-3-apps, 2020.

[37] OpenAI. Chatgpt: Language models for task-oriented dialogue. https://openai.com/blog/chatgpt/, 2021.

[38] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[39] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[40] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

[41] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.

[42] SageMaker. Sagemaker. https://docs.aws.amazon.com/sagemaker/index.html.

[43] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*, 2022.

[44] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[45] Dharma Shukla, Muthian Sivathanu, Srinidhi Viswanatha, Bhargav Gulavani, Rimma Nehme, Amey Agrawal, Chen Chen, Nipun Kwatra, Ramachandran Ramjee, Pankaj Sharma, Atul Katiyar, Vipul Modi, Vaibhav Sharma, Abhishek Singh, Shreshth Singhal, Kaustubh Welankar, Lu Xun, Ravi Anupindi, Karthik Elangovan, Hasibur Rahman, Zhou Lin, Rahul Seetharaman, Cheng Xu, Eddie Ailijiang, Suresh Krishnappa, and Mark Russinovich. Singularity: Planet-scale, preemptive and elastic scheduling of ai workloads, 2022.

[46] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large {DNNs}. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 497–513, 2023.

[47] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[48] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[49] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, TS Eugene Ng, and Yida Wang. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 364–381, 2023.

[50] Baodong Wu, Lei Xia, Qingping Li, Kangyu Li, Xu Chen, Yongqiang Guo, Tieyao Xiang, Yuheng Chen, and Shigang Li. Transom: An efficient fault-tolerant system for training llms. *arXiv preprint arXiv:2310.10046*, 2023.

[51] Yidi Wu, Kaihao Ma, Xiao Yan, Zhi Liu, Zhenkun Cai, Yuzhen Huang, James Cheng, Han Yuan, and Fan Yu. Elastic deep learning in multi-tenant gpu clusters. *IEEE Transactions on Parallel and Distributed Systems*, 33(1):144–158, 2022.

[52] Lei Xie, Jidong Zhai, Baodong Wu, Yuanbo Wang, Xingcheng Zhang, Peng Sun, and Shengen Yan. Elan: Towards generic and efficient elastic training for deep learning. In *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, pages 78–88, 2020.

[53] Andy B Yoo, Matt A Jette, and Mark Grondona. Slurm: Simple linux utility for resource management. *Job Scheduling Strategies for Parallel Processing*, pages 44–60, 2003.

[54] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[55] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 559–578, 2022.